JTS Topology Suite

Technical Specifications

Version 1.4 / 1.4.1, rebuilt

Base text: 2003 · rebuild: 16 August 2026

This document is the design specification for the classes, methods and algorithms implemented in the JTS Topology Suite. The 2003 chapter plan is kept. Sections that would now lie about the spatial model, WKT/WKB, overlay, or Hausdorff are amended against the code on this tree.

Package names in this tree are `org.locationtech.jts`. The 2003 copies used `com.vividsolutions.jts`. LocationTech JTS remains the upstream project; this fork is not described as if it were already upstream.

Document change control

Revision	Date	Author	Change
1.3	31 Mar 2003	M. Davis	Updated to cover changes in JTS 1.3.
1.4	17 Oct 2003	J. Aquino	CoordinateSequences and the user-data field.
1.4.1	2003	M. Davis	Fixed definition of contains.
1.4-rebuild	16 Aug 2026	This tree	LaTeX rebuild. SFS Curve / MultiCurve / MultiSurface are no longer abstract-only. WKB 8–12, OverlayNGCurve, hulls, and the two D-HF pairs are recorded.

Contents

1 Overview

The JTS Topology Suite is a Java API that implements a core set of spatial data operations using an explicit precision model and robust geometric algorithms. JTS is intended to be used in the development of applications that support the validation, cleaning, integration and querying of spatial datasets.

JTS attempts to implement the OpenGIS Simple Features Specification (SFS) as accurately as possible. Where the SFS is unclear or omits a specification, JTS chooses a reasonable and consistent alternative. Differences from and elaborations of the SFS are documented here.

The 2003 spec stopped at linear SFS. This tree adds opt-in ISO 19125-2 / SQL-MM curve types in `jts-curve`. The detailed class hierarchy remains JavaDoc. This rebuild records the contract a reader of the 2003 PDF would otherwise get wrong.

2 Other resources

- OpenGIS Simple Features Specification for SQL Revision 1.1 (SFS) — master specification for the linear spatial model and predicates.
- OGC Simple Features Access 1.2.1 / ISO 19125-2 — curve types and WKB codes 8–12.
- *JTS Developer's Guide* and *TestBuilder & TestRunner User Guide* (siblings in this directory), rebuilt against the same tree.

3 Design goals

The 2003 goals are still the goals of JTS core:

- Conform to the SFS as accurately as possible, consistent with a correct implementation.
- Follow Java conventions (`getX` / `setX`, `isX`, lowercase method names).
- Support a user-defined precision model; algorithms are robust under it.
- Return topologically and geometrically correct results within that precision model wherever possible.
- Correctness is the highest priority; space and time efficiency is important but secondary.
- Fast enough for production. Algorithms and code should stay clear.

The curve module adds a further constraint that is not in the 2003 spec: a closed-form curve path that would be slower than densify-then-core is refused, and the linear path runs instead.

4 Terminology

Term	Definition
Coordinate	A point in space exactly representable under the precision model.
Node	A point where two lines intersect. Not necessarily a representable coordinate.
Noding	Computing the nodes where geometries intersect.
Robust computation	Numerical computation guaranteed to return the correct answer for all inputs.
SFS	OGC Simple Features Specification (linear).
SQL-MM / SFA 1.2.1	Extended types: CircularString, CompoundCurve, CurvePolygon, MultiCurve, MultiSurface.
Closed form	A cheap shape check that answers exactly. On this tree: certified discs, the single-arc hull, the compound-curve hull of circular-plus-straight members, and the two D-HF pairs.
Chord / densify	The fallback: replace arcs with LineString segments. Not a laser.
Vertex	A corner point stored explicitly.

5 Notation

Items that adhere to the SFS are marked (SFS). Items that elaborate or differ are marked (JTS). Items that exist only on this tree's curve module are marked (jts-curve).

6 Java implementation

Where SFS coding style differs from Java, JTS follows Java: `boolean` instead of Integer-as-boolean; method names start with a lowercase letter; get/set prefixes for JavaBeans. The 2003 package root `com.vividsolutions.jts` is `org.locationtech.jts` on this tree. That is the LocationTech package rename, not a claim that this fork is LocationTech upstream.

7 Computational geometry issues

7.1 Precision model

JTS lets the client state how many bits of precision are assumed in input coordinates and maintained in computed coordinates. Two basic types are supported: Fixed (coordinates on a uniform grid, scale factor) and Floating (IEEE-754 double or single). Input routines supplied with JTS round to the precision model. Incompatible precision models are not reconciled.

In the Fixed model, coordinates fall on a discrete grid. The grid size is the inverse of the scale factor:

$$\text{jtsPt}.x = \text{round}(\text{inputPt}.x \cdot s)/s, \qquad \text{jtsPt}.y = \text{round}(\text{inputPt}.y \cdot s)/s.$$

Precise coordinates are represented internally as IEEE-754 doubles (53 bits of precision).

Floating Double uses the full Java double. Floating Single rounds computed coordinates to single precision (for example when the destination is Java2D).

JTS does not implement an exact (rational) precision model.

7.2 Constructed points and dimensional collapse

Overlay may construct points that are not input vertices. Intersection coordinates can require twice the input precision; JTS rounds them to the PrecisionModel. Rounding can produce dimensional collapse. JTS forms the lower-dimension Geometry that results.

7.3 Robustness and numerical stability

Fundamental predicates (orientation, line intersection, point-in-ring) use robust algorithms. The binary-predicate algorithm is completely robust. Overlay and buffer are engineered to return correct answers in the majority of cases; they are not a substitute for exact arithmetic. Line-intersection inputs are normalized toward the origin to preserve bits.

7.4 Performance — monotone chains

Intersection detection uses in-memory spatial indexes and monotone chains: sequences of segments whose direction vectors lie in one quadrant. Segments inside one chain do not intersect; envelopes of contiguous subsets are the envelopes of the endpoints, so binary search applies. This is unchanged for linear Geometry. It is not a circular noder.

8 Spatial model (linear, kept)

The 2003 design decisions stand for linear Geometry: repeated points are allowed (SFS); LineStrings may be non-simple (SFS); polygon rings may not self-touch; polygon rings may mutually touch at a single point.

8.1 Geometric definitions (linear)

Every Geometry carries a PrecisionModel and an optional user-data field. Empty geometries are allowed; a generic empty is an empty GeometryCollection. The dimension of a heterogeneous GeometryCollection is the maximum dimension of its elements.

Curve (SFS, 2003). The 2003 spec defined SFS Curve as a non-degenerate linear curve: at least two points, no two consecutive points equal. The only concrete curve was LineString / LinearRing. That paragraph is still true of core.

MultiCurve (SFS, 2003). Boundary uses the Mod-2 rule: a point is on the boundary iff it is on the boundary of an odd number of elements. The 2003 figures showing that this can disagree with point-set topology still apply to MultiLineString.

LineString / LinearRing / Polygon / MultiPolygon. Unchanged. LineStrings may self-intersect. LinearRings have at least 4 points (including the repeated close), are simple, and may be CW or CCW. Polygon shells and holes are LinearRings; holes may touch the shell or another hole at a single point only; interiors are connected.

8.2 Support classes

Coordinate, CoordinateSequence, Envelope, IntersectionMatrix, GeometryFactory, CoordinateFilter, GeometryFilter remain as in 2003. GeometryFactory on this tree also has `createCircularString` / `createCompoundCurve` / `createCurvePolygon` / `createMultiCurve` / `createMultiSurface` stubs that throw unless the factory is a `CurveGeometryFactory` (jts-curve).

9 Spatial model — curve types on this tree

On this tree the concrete SFA/SQL-MM curve types live in the `jts-curve` module (`org.locationtech.jts.geom.cu` `CircularString`, `CompoundCurve`, `CurvePolygon`, and the collections `MultiCurve` and `MultiSurface`. Construction is opt-in: the default `GeometryFactory` throws on the curve `create*` methods. Use `CurveGeometryFactory`. Constructors do not linearise.

9.1 CircularString (jts-curve)

A CircularString is a LineString whose consecutive triples (start, mid, end) are circular arcs. OGC SFA requires an odd point count ≥ 3. It is not a LineString of those control chords. `toLinear(tolerance)` densifies explicitly. The class is not constructed by the default GeometryFactory.

9.2 CompoundCurve (jts-curve)

A CompoundCurve is a LineString whose members are LineString and/or CircularString pieces, end-to-start connected. Member structure is preserved on copy and on `CurveWKTWriter` / `CurveWKBWriter`. A clothoid may appear as a non-leading member when read by `CurveWKTReader`; core `WKTReader` still fails on the `CLOTHOID` keyword. Clothoid membership is extras, not a closed-form construction.

9.3 CurvePolygon (jts-curve)

A CurvePolygon is a Polygon whose rings may be CircularString, CompoundCurve, or LinearRing. A full-circle CircularString shell is a certified disc. Disc area, envelope, and disc-buffer are closed form on that shape. A CurvePolygon that is not a certified disc is not promoted to a laser.

9.4 MultiCurve and MultiSurface (jts-curve)

MultiCurve extends MultiLineString; MultiSurface extends MultiPolygon. They are the SQL-MM collections (WKB 11 and 12). A MultiSurface of one certified disc unwraps to that disc for the D-HF disc pair. They are not a general overlay noder.

9.5 What this section does not add

No public noder. Constructors do not linearise. This fork is not described as LocationTech upstream. Triangle / PolyhedralSurface / TIN exist in the same module as additional ISO types; they are not the curve overlay stack.

10 Basic geometric algorithms

The 2003 primitives remain the linear primitives: point-line orientation, line intersection test and computation, point-in-ring, ring orientation. They operate on coordinates and segments. They do not node circular arcs. Arc/circle predicates used by certified-disc paths live in `jts-curve` (package-private helpers) and are not a public noder.

11 Topological computation and overlay

The 2003 spec described topology graphs, labels, the Relate algorithm, and the overlay algorithm that built a noded graph and extracted result edges. That is the historical linear overlay. Production linear overlay on this tree is OverlayNG.

The curve overlay type is `OverlayNGCurve`. It lives in `jts-curve` (`org.locationtech.jts.operation.overlayng.`
There is no public noder. Closed-form answers are limited to certified discs. Anything else densifies and delegates to core OverlayNG. `Geometry.intersection` / `union` / `difference` / `symDifference` on a curve type are not a general circular overlay.

Relate / DE-9IM on a certified disc vs Point, LineString, hole-free Polygon, or a second disc has closed-form cases in the curve module. Half-disc and general CompoundCurve relate that have no kit fall through to linearise. That fall-through is not a general arc-aware relate.

12 Binary predicates

Equals, Disjoint, Intersects, Touches, Crosses, Within, Contains, Overlaps keep their 2003 / SFS meanings on linear Geometry. Two areas never Crosses. The 2003 contains fix (1.4.1) stands. On curve operands, a predicate is exact only when a certified path answers; otherwise it sees chords or an explicit `toLinear` copy.

13 Spatial analysis methods

Constructive methods (Buffer, ConvexHull) and set-theoretic methods (Intersection, Union, Difference, SymDifference) keep their 2003 point-set definitions. Representation of computed geometries is still constrained by the precision model.

Buffer (linear). Unchanged: Minkowski sum with a disc, approximated by quadrant segments, end-cap styles as in the Developer's Guide.

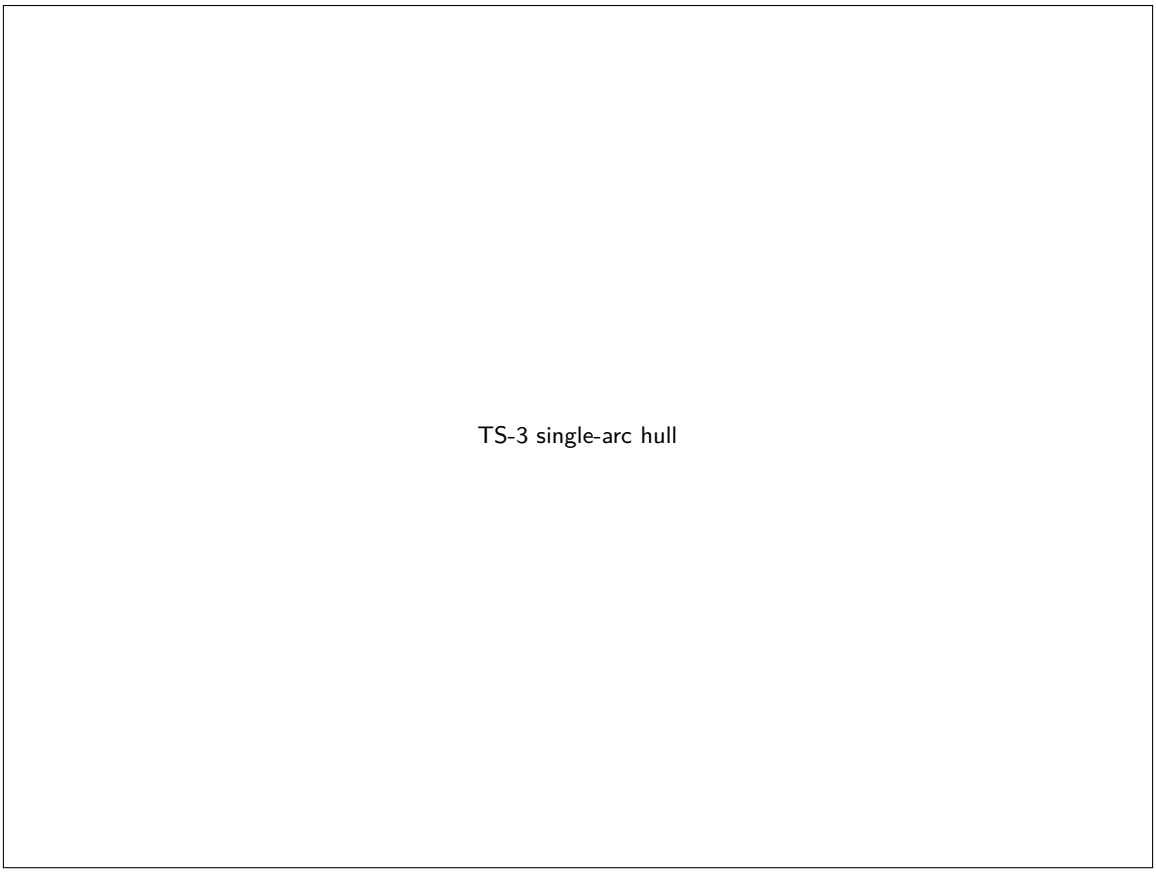
Buffer (curve, jts-curve). Certified disc → disc of radius $r + d$ or empty. Open-arc buffer is BufferOp on chords — not a closed form.

ConvexHull (linear). Unchanged: smallest convex polygon, or a lower-dimension Geometry if fewer than 3 hull points; no three consecutive hull points collinear.

ConvexHull (curve, jts-curve). A certified disc's hull is the disc (radius-5 disc: area 25π). A single-arc hull is the slice bounded by the arc and its chord (a CurvePolygon). The compound-curve hull of circular-plus-straight members (H-CC), including a stadium, keeps the exposed arcs as CircularString members of a CurvePolygon. Any other curve uses the linear hull of `toLinear` — a fallback, not a fail.

Inscribed and empty circles (jts-curve). `MaximumInscribedCircle` on a certified disc and on a stadium takes a closed form. `LargestEmptyCircle` on legal curve obstacles is present. These are constructions, not a general circular noder.

TS-2 disc hull



14 Other methods

Boundary, isClosed, isSimple, isValid keep the 2003 tables for linear types. JTS does not validate on construct (efficiency). IsValidOp reports the nature and location of a failure. Curve validation is best-effort structural (odd CircularString count, member connectivity); it is not a full SQL-MM validator.

15 Well-Known Text and Well-Known Binary

15.1 WKT — linear (kept)

SFS §3.2.5. JTS follows the `MULTIPOINT (x y, x y)` example form, not the parenthesized-pairs grammar. JTS extends WKT with `LINEARRING`. `WKTReader` is parameterized by a GeometryFactory (precision and SRID). `WKTWriter` rounds to the precision model.

15.2 WKT — curve (jts-curve)

Core `WKTReader` still rejects `CIRCULARSTRING` and the other SQL-MM keywords. Read them with `CurveWKTReader`. Write structured members with `CurveWKTWriter`. Core `WKTWriter` already emits

the subclass keyword via `getGeometryType()` for a flat CircularString; composite types need the curve writer to keep member tags.

Added tagged text (SQL-MM): `CIRCULARSTRING`, `COMPOUNDCURVE`, `CURVEPOLYGON`, `MULTICURVE`, `MULTISURFACE`. `CLOTHOID` is recognised only by `CurveWKTReader`, and only as a non-leading `COMPOUNDCURVE` member. That path is extras, not a laser. Core `WKTReader` still fails on those keywords.

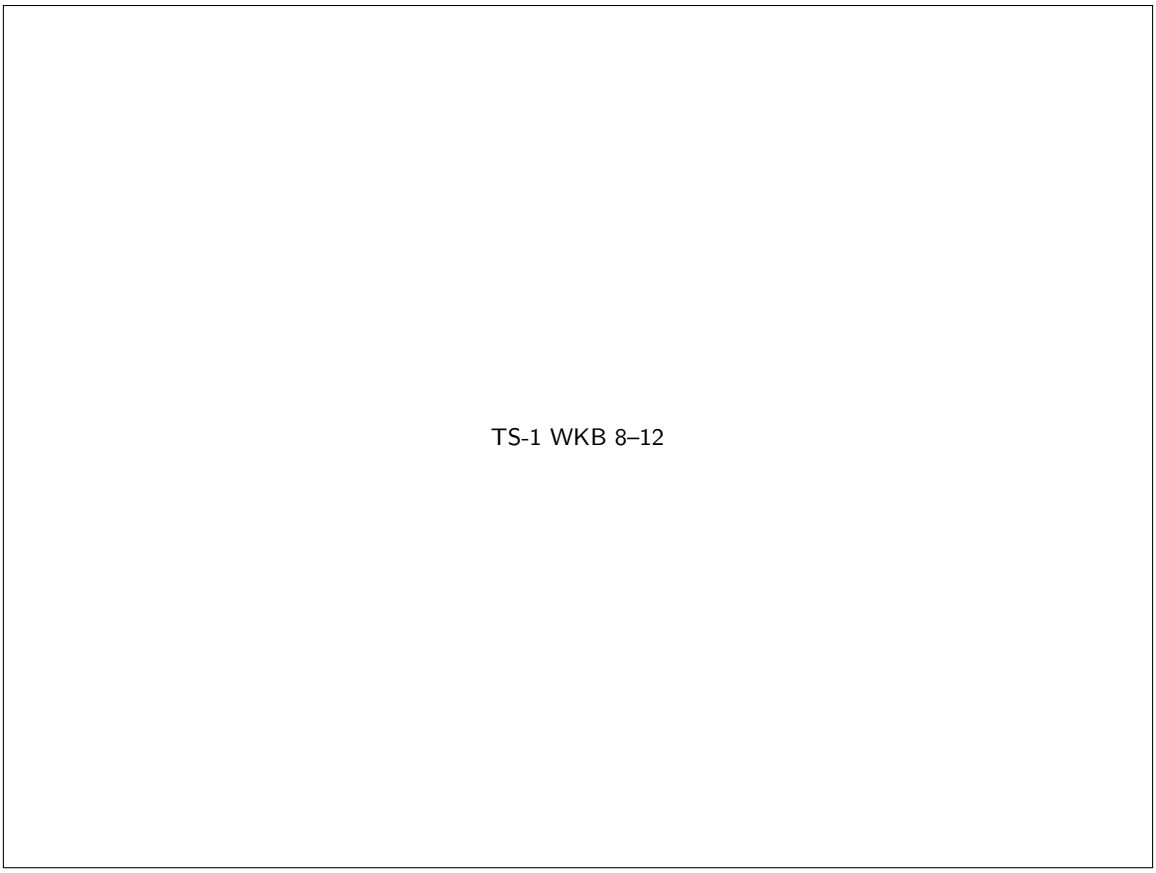
15.3 WKB

The 2003 spec did not list WKB codes 8–12. On this tree:

Code	Type	Read (core WKBReader)	Write
1–7	Point ... GeometryCollection	Yes, default factory	WKBWriter
8	CircularString	Yes; `factory.createCircularString`	CurveWKBWriter. Bare WKBWriter emits 2
9	CompoundCurve	Yes; `factory.createCompoundCurve`	CurveWKBWriter. Bare WKBWriter emits 2
10	CurvePolygon	Yes; `factory.createCurvePolygon`	CurveWKBWriter. Bare WKBWriter emits 3
11	MultiCurve	Yes; `factory.createMultiCurve`	CurveWKBWriter. Bare WKBWriter emits 5
12	MultiSurface	Yes; `factory.createMultiSurface`	CurveWKBWriter. Bare WKBWriter emits 6

WKB type codes 8–12 are first-class in core `WKBReader` and `WKBConstants`. Construction still requires a curve-capable factory; otherwise the reader throws. `CurveWKBWriter` emits codes 8–12. A bare `WKBWriter` still walks the parent `instanceof` ladder and writes a CircularString as LineString (code 2) and a CurvePolygon as Polygon (code 3). Default and export WKB paths route through `CurveWKBWriter`.

Default GeometryFactory throws on the `create*` curve methods, so `new WKBReader().read(type8)` fails with a factory-required ParseException. `new WKBReader(new CurveGeometryFactory())` or `CurveWKBReader` succeeds.



16 Discrete Hausdorff distance

The 2003 spec did not define Hausdorff. On this tree the public class is `org.locationtech.jts.algorithm.distanc`

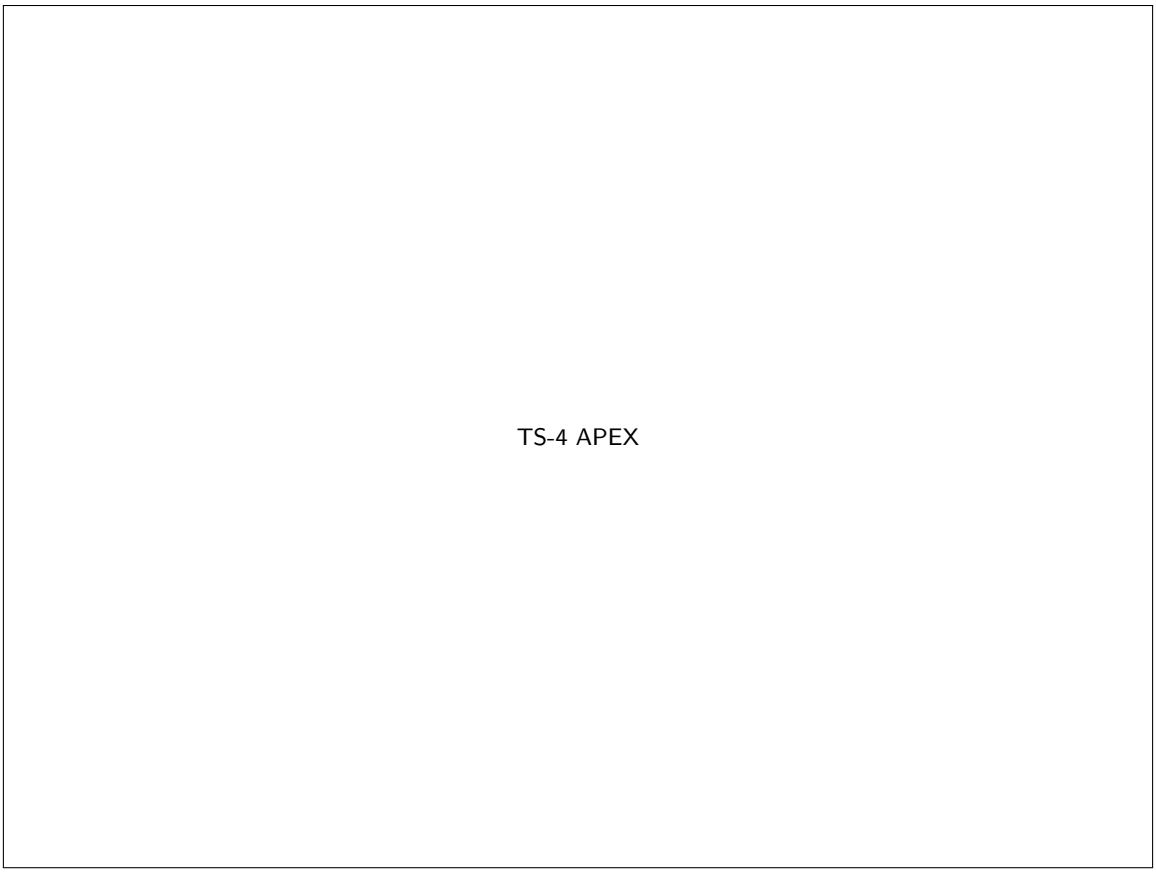
Public `DiscreteHausdorffDistance` (commit `0ca71b`) has a closed form for two pairs only:

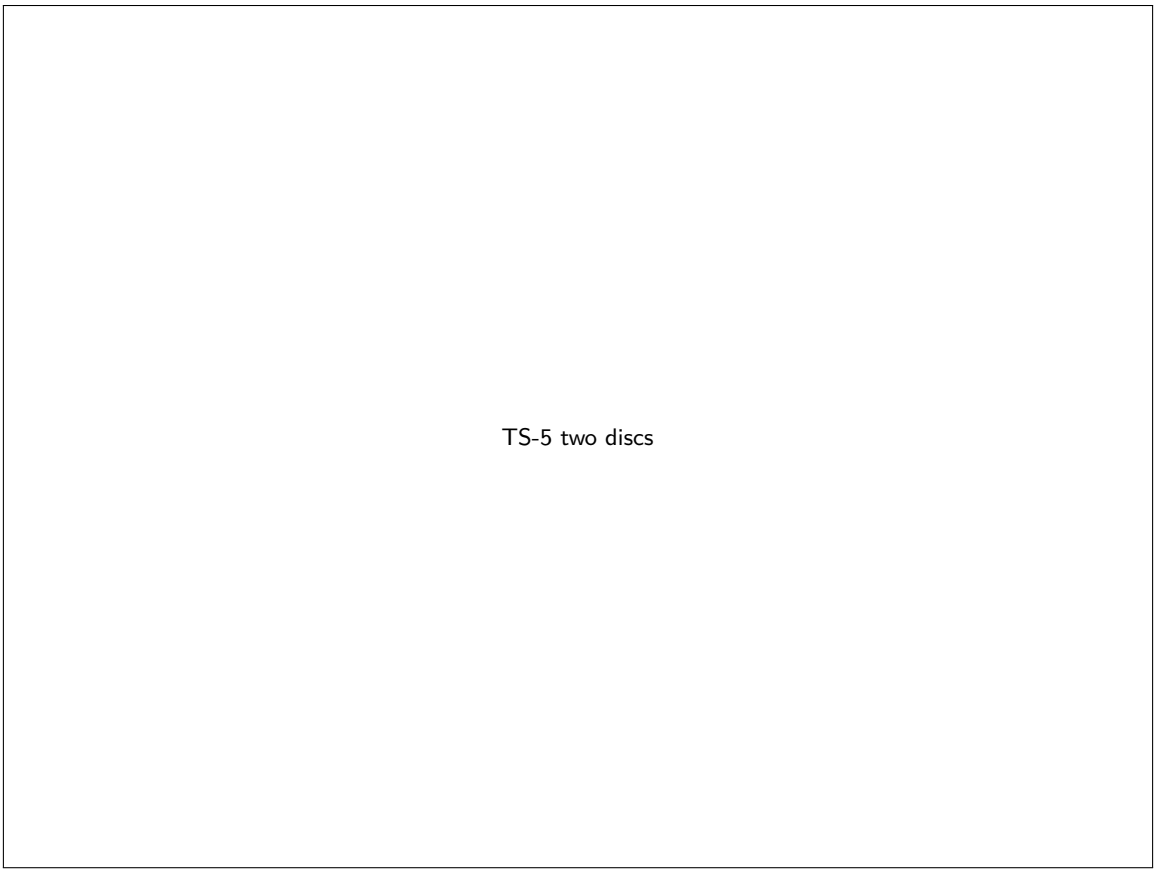
1. A single-arc CircularString toward a single-segment LineString, apex $\sqrt{949}/6 - 7/6 \approx 3.967641$ for `CIRCULARSTRING (0 0, 2 3, 10 0)` vs `LINESTRING (0 0, 10 0)`.
2. Two circular discs (full-circle CurvePolygon). A MultiSurface unwrap of one disc is the same disc pair.

The exact path skips densify; densify is not the laser. For every other pair the public API still sees vertices / chords. Tests keep `fail()` on the general case.

`DensifyFrac` still interpolates chords on the fallback path. Calling `distance(..., densifyFrac)` on a certified pair does not replace the closed form with densify; the exact path skips densify. Do not document densify as the laser.

Fréchet (`DiscreteFrechetDistance`) remains open as a general curve metric. This specification does not claim a Fréchet laser.


TS-4 APEX



17 Licence

JTS is dual-licensed under the Eclipse Public License 2.0 and the Eclipse Distribution License 1.0. See `LICENSE_EPLv2.txt`, `LICENSE_EDLv1.txt`, and `LICENSES.md`. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

18 References

SFS OpenGIS Simple Features Specification For SQL Revision 1.1. Open GIS Consortium, Inc. OpenGIS project Document 99-049.

SFA OpenGIS Implementation Standard for Geographic information — Simple feature access — Part 1: Common architecture, 1.2.1 / ISO 19125.

Ava97 F. Avnaim, J-D. Boissonnat, O. Devillers, F. Preparata and M. Yvinec. Evaluating signs of determinants using single-precision arithmetic. *Algorithmica* 17:111–132, 1997.

Bri98 A. Brinkmann, K. Hinrichs. Implementing exact line segment intersection in map overlay. SDH 1998, pp. 569–579.

Sch97 Stefan Schirra. Precision and robustness in geometric computations. In *Algorithmic Foundations of Geographic Information Systems*, LNCS 1340, Springer, 1997.

End of Technical Specifications. Named figure holes are empty 4:3 slots (target 1600×1200, full TestBuilder window). No clothoid DOI is cited.